Room Number _____

Seat Number _____

Student Number |__|__|__|__|__|__|__|__|__|

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

THE UNIVERSITY OF SYDNEY

SAMPLE EXAMINATION PAPER

Information Technologies

SAMPLE EXAMINATION

Semester 1 - Main, 2018

Systems Programming

EXAM WRITING TIME: 2.5 hours

READING TIME: 10 minutes

EXAM CONDITIONS:
This is a CLOSED book examination - no material permitted

During reading time - writing is not permitted at all

MATERIALS PERMITTED IN THE EXAM VENUE:
(No electronic aids are permitted e.g. laptops, phones)

Calculator - non-programmable

MATERIALS TO BE SUPPLIED TO STUDENTS:
1 x 8-page answer book

INSTRUCTIONS TO STUDENTS:
Write all your answers in the location specified by the question. You should clearly indicate the question being answered in the answer booklet.

For Examiner Use Only

Q	Mark
1	
2	
3	
4	
5	
6	
7	
8	
9	

Total _____

Please tick the box to confirm that your examination paper is complete. ☐

This document is available on studocu

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Multiple choice. 1 mark each unless stated otherwise

Question 1 What is the difference between a C string and any `char` array?

- ☐ C string uses the last character to indicate the end of the string.
- ☐ `char` array does not store any quote characters
- ☐ C string is stored in static memory space
- ☐ no difference

Question 2 What is the value of the following code?

```
int c = "QUOKKA"[3];
char k[] = { 0, 98, 99, 98, 0 };
k[0] = c;
printf("%s\n", (char*)k);
```

- ☐ `Kbcb`
- ☐ No output. Segmentation Fault
- ☐ `K`
- ☐ Cannot compile

Question 3 Which of the following is valid C code syntax?

- ☐
```
String str = "12345";
int i = 0;
while ( i++ < str.length());
```
- ☐
```
str = "12345";
for (c in str):
continue;
```
- ☐
```
char *str = "12345";
while (*str++);
```
- ☐
```
String str = "12345";
int i = 0;
for (int i = 0; i < str.length();
++i);
```
- ☐
```
char *str = "12345";
while (i++ && i < strlen(str); ) continue
```
- ☐ *None of these answers are correct.*

Question 4 ♣ - Multiple answer. Worth 2 marks

Which statements are true about the C preprocessor?

- ☐ It can define a macro
- ☐ It can be used for conditional compilation
- ☐ It cannot produce errors
- ☐ It is a program run after linking
- ☐ `#include` is a command that will search the current directory
- ☐ It performs text processing
- ☐ It identifies the command `#define`
- ☐ It is only available in UNIX or Linux systems

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Question 5 (Memory) (15 marks)

Write the memory contents after execution of all statements up to line 17. Write the names of each variable, its address and any data associated with them in the appropriate box.

```
1  extern void *lerp(void*);
2  struct tee {
3      int x;
4      void* m;
5      void *p;
6  };
7  struct tee t;
8  int main( int argc, char **argv ) {
9      int a = 73;
10     static void *(*flerp)(void*) = lerp;
11     t.x = 51;
12     t.m = &a;
13     struct tee s = t;
14     s = *((struct tee*)(flerp((void*)&t)));
15     t.p = &s;
16     printf("%s\n", (const char*)&a);
17     // show snapshot of memory at this statement
```

The starting address for each segment of memory has been provided for you. For the purpose of this exam, you should assume that memory is allocated positively from 0x0000 to 0xFFFF.

`sizeof(char)` is 1 byte and `sizeof(char *)` is 4 bytes. The ASCII value of `'A'` is 65. Assume memory does not align to nearest word size.

Consider these two lines of code, the memory contents may look like ⟶

```
char a = 5;
char *b = &a;
```

Sample memory
0x7000 a 5
0x7001 b 0x7000

Program code
0x2000

Static/Global
0x3000

Heap
0x4000

Stack
0x7000

This document is available on studocu

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Question 6 (Programming) (15 marks)

Write a C function

```
int best_argset(int *argc, char ***argv)
```

`best_argset` function returns the index of one **argument set** with the highest number of characters. An array of arguments is defined in the same memory format as the C main function `int argc, char **argv`, where `argc` is a single integer representing the number of arguments and `argv` is the array of C strings. This is known as the **argument set**.

`best_argset` function accepts an array of argument sets, where `argc` is an array of integers, `argv` is an array of an array of C strings. The argument set with the highest number of characters has an index from the input. This index is the return value of the function.

On success, the index of the argument set with the highest number of characters is returned. If there is more than one argument set discovered to have the highest, only one is returned and it can be any. `-1` is returned If any parameter is `NULL`.

Example of input data and return value:

	index	argc[index]	argv[index][0]	argv[index][1]	argv[index][2]
e.g. 1	0	3	"abc"	"def"	"ghi"
	1	1	"ruck rover"		
	2	3	"./a.out"	"launch"	"5"

Returns 2

The following function may be useful:

```
size_t strlen(const char *str);
```

The function `strlen()` calculates the length of the string pointed to by `str`, excluding the terminating null byte.

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

This document is available on studocu

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Question 7 (Memory) (20 marks)

The following code implements an operation on a doubly linked list to add one element.

```
struct node {
    struct node *next;
    struct node *prev;
    int* value;
};

// takes pointer to pointer to value and performs operations on value
void add_value(struct node** valuep, int* v){
    //printf("addnull %d v %d"
    if(valuep == NULL){
        struct node **v = malloc(sizeof **v);
        valuep = v;

    }
    if( (*valuep) == NULL ){
        //printf("valuep null %d",*v);
        // set new value

        struct node *val = malloc(sizeof *val);
            *valuep = val;
            val -> next = NULL;
            val -> prev = NULL;
            val -> value = v;
    } else if( (*valuep) -> next == NULL){
            //printf("valuep next null %d v %d\n",*(*valuep)->value,*v);
        struct node *val = malloc(sizeof *val);
            (*valuep)->next = val;
            val -> next = NULL;
            val -> prev = *valuep;
            val -> value = v;
            //printf("valuep %d v %d\n",*(*valuep)->value, *v);
            //printf("val    %d v %d\n",*val->value   , *v);
            //ent -> values = value;
            //add_value(ent, value);

    } else {
        add_value( &(*valuep)->next, v);
    }

}
...
```

Questions

(a) Does the code compile without errors (Yes / No)? **4 marks**

(i) If you answered YES, what is wrong with the function `add_value`? annotate the code with brief comments and arrows, circles etc.

(ii) If you answered NO, what syntax needs to be corrected to make the code compile? annotate the code with brief comments and arrows, circles etc.

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Error message - 4 marks

Any potential syntax errors were removed, and the program was run with the test data. The following message is shown:

```
ASAN:DEADLYSIGNAL
- =================================================================
- ==6==ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000 (pc 0x0000004ea9e9 bp 0x7f
-     #0 0x4ea9e8 in add_value /home/linked-data.c:53:26
-     #1 0x4ec40c in main /home/linked-data.c:589:7
-     #2 0x7f512bd6d82f in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2082f)
-     #3 0x4185e8 in _start (/home/linked-data+0x4185e8)
- AddressSanitizer can not provide additional info.
- SUMMARY: AddressSanitizer: SEGV /home/linked-data.c:53:26 in add_value
- ==6==ABORTING
```

Describe what the error means, what offending value is present in the stack, and how to fix it.

Memory size - 6 marks

(b) **2 marks** Describe what the operator `sizeof` is achieving in the context of the code on lines 40, 48, and 55.

(c) **1 mark** Given that this program was run on a 64 bit memory addressable CPU. What are the values of `sizeof` on lines 40, 48, and 55?

(d) **2 marks** Assume the function performs it's intended purpose of adding one element to the doubly linked list. If 5 elements were inserted in succession, list all the stack variables associated when inserting the 5th element (the point of insertion can be considered as lines 41, 52, or 59).

(e) **1 mark** Assume the function performs it's intended purpose of adding one element to the doubly linked list. What is the amount of heap memory allocated after 4 new elements are inserted successfully.

This document is available on studocu

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Delete - 6 marks

Write the entire function `delete_value()`, include both the prototype and function body. `delete_value()` removes a specific value, `int* v`, from the doubly linked list. It will return a pointer to a node struct that was removed. If there is more than one value, then only one, any chosen one, is removed and is returned. `NULL` is returned if 1) there is no such element with the value `v`, or; 2) the doubly linked list is `NULL`, or 3) the value `v` is `NULL`.

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Question 8 (Programming) (30 marks)

Write in ANSWER BOOKLET

Question 8A Sequential part - 16 marks

This question considers a 2D grid of floating point numbers with dimensions `width` and `height`. Find the position of this grid that has the highest magnitude average. This function computes the sum of the neighbouring values in the 2D grid (NorthWest, North, NorthEast, East, SouthEast, South, SouthWest, West) plus the centre and divides this by 9. Neighbours outside the dimensions of the array are treated as value zero. The grid is given as a 1D array which stores the 2D grid positions row-major order. `NULL` is returned in `found_x` or `found_y` if there is an error. The error can arise if the input array is `NULL`, dimensions are ≤ 0, or `found_x` or `found_y` are `NULL` values themselves.

The 1D array sequence: 1, 2, 3, 4, 5, 6, 7, 8, 9 represents the grid:

1	2	3
4	5	6
7	8	9

$$f(x,y) = \frac{1}{9} \sum_{\substack{x-1<i<x+1 \\ y-1<j<y+1}} |array(i,j)|$$

Write the code needed for the following function prototype to work. You may write additional helper functions if needed.

```
// pre: w  > 0 h > 0
// returns the position of the highest magnitude average of the array
// in variables found_x and found_y
void get_hma(float *array, int w, int h, int *found_x, int *found_y)
```

Question 8B Parallel part - 7 marks

Consider a parallelisation of this problem. Describe in words how you would use 2 or more threads to gain a speedup > 1.
In your answer, you should explain if the approach is task parallel, data parallel, or both and also provide a drawing of a task-dependency graph that supports your description.

Question 8C Parallel part - 7 marks

Write a parallel solution using exactly 4 threads to gain a speedup > 1. Write your solution with notation that is *similar* to pthreads. `thread_create()`, `thread_join()`, `lock(mutex)`, `unlock(mutex)`. `sem_wait(s)`, `sem_post(s)`, `sem_init(s, value)`.

WRITE YOUR SOLUTION FOR THIS QUESTION IN YOUR ANSWER BOOKLET

This document is available on studocu

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Question 9 (Concurrency) (15 marks)

Write in ANSWER BOOKLET

You are to create an interprocess communication mechanism.

A program's initial process, `P0`, creates and maintains a stack data structure. Many child processes are created thereafter and they rely on the same stack data structure of the parent process `P0`.

Only the parent process `P0` can modify the stack contents. Any child process can operate on the stack by communicating with `P0`.

There are two types of child processes:

- Worker - performs operations with the stack if is is not full or not empty.
- Cleaner - performs operations with the stack when it is full or when it is empty. The cleaner aims to make the stack non-empty and non-full.

Cleaner processes cannot operate on the stack while Worker processes operate and vice versa.

The stack operations are as follows:

```
// places the integer x on top of the stack
// Otherwise, err is set to non-zero value if the stack is full
void push( struct stack *, int x, int *err );

// removes and returns the integer on top of the stack
// Otherwise, err is set to non-zero value if the stack is empty
int pop( struct stack *, int *err );

// returns the integer on top of the stack
// Otherwise, err is set to non-zero value if the stack is empty
int peek( struct stack *, int *err );

// returns non-zero value when empty
int is_empty(  );
```

Describe interprocess communication necessary to keep the stack synchronised for `N` child processes, where child processes can be either Worker or Cleaner types.

You can describe your answer with pseudocode, or C code (most accurate). If your answer is not clear, marks can be deducted.

WRITE YOUR SOLUTION FOR THIS QUESTION IN YOUR ANSWER BOOKLET

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)

Useful functions for your reference:

Setting the signal handler

```
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);

Sets the disposition of the signal signum to handler, which is either SIG_IGN, SIG_DFL, or the
     address of a  programmer defined function (a "signal handler").

signal()  returns the previous value of the signal handler, or SIG_ERR on error.
```

Sending the signal

```
int raise(int sig);
Sends a signal to the calling process or thread.
     If the signal causes a handler to be called, raise() will return only after the signal
     handler has returned.

raise() returns 0 on success, and nonzero for failure.
```

Setting up the pipe

```
int pipe(int pipefd[2]);
creates a pipe, a unidirectional data channel that can be used for interprocess communication.
     The array pipefd is used to return two file descriptors referring to the ends of the pipe.
     pipefd[0] refers to the read end of the pipe. pipefd[1] refers to the write end of the pipe.

On success, zero is returned. On error, -1 is returned.
```

End of the Exam

This document is available on studocu

Downloaded by sdf werfwqqfr (kenny031031@gmail.com)